Tổng hợp và biên soạn: ……

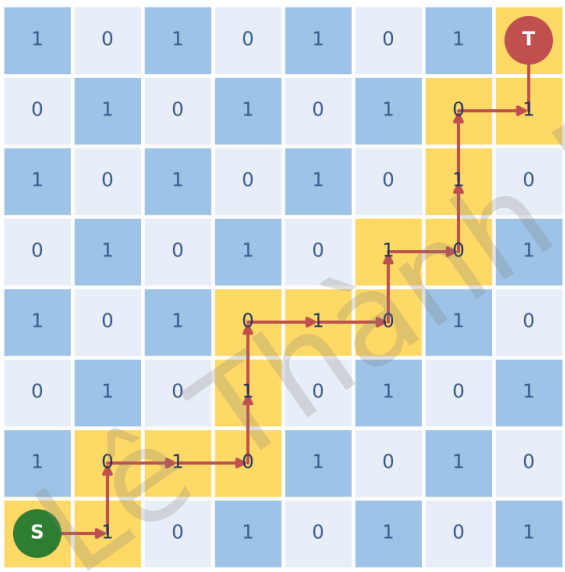
TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

PARITY

TÍNH CHẴN LẺ TRONG TIN HỌC

Từ cơ bản đến nâng cao

TẬP 1 · LÝ THUYẾT



⊕	0	1
0	0	1
1	1	0

chẵn ⇄ 0

lẻ ⇄ 1

1 + 1 ≡ 0 (mod 2)

Lê Thành Nam

$$p(a + b) = p(a) \oplus p(b)$$

Chương 1. NỀN TẢNG TƯ DUY PARITY
Chương 2. PARITY TRÊN DÃY VÀ XÂU
Chương 3. PARITY TRONG TỔ HỢP VÀ ĐỒ THỊ
Chương 4. PARITY NÂNG CAO: QUY HOẠCH ĐỘNG, QUAN HỆ XOR, GF(2)

LỜI NÓI ĐẦU

Trong đề thi học sinh giỏi Tin học, tính chẵn lẻ (parity) hiếm khi được gọi đúng tên. Đề bài nói về bóng đèn, robot, quân cờ, những phát biểu "cùng đội" hay "khác đội", những lần đổi chỗ… nhưng đằng sau rất nhiều câu phân loại là cùng một ý: câu hỏi cuối cùng chỉ cần một bit thông tin. Học sinh không nhìn ra điều đó sẽ lao vào duyệt hàng tỉ trạng thái; học sinh nhìn ra sẽ giải bài trong vài dòng.

Nếu đây là lần đầu tiên em gặp từ parity, đừng bắt đầu bằng định nghĩa `x % 2`. Một định nghĩa đúng chưa chắc tạo ra được tư duy giải bài. Điều ta cần là một câu hỏi khác: vì sao có những bài toán khổng lồ nhưng đáp án lại chỉ cần một bit? Từ câu hỏi ấy, cuốn sách sẽ lần lượt dẫn em đi qua XOR, bất biến, tô hai màu, prefix parity, parity mask, DSU parity và cuối cùng là đại số tuyến tính trên GF(2).

Bộ tài liệu được viết để giúp em hình thành cách suy nghĩ ấy một cách có hệ thống. Cuốn sách này là:

• Tập 1: Lý thuyết gồm 13 bài học chia thành 4 chương: Nền tảng tư duy parity; Parity trên dãy và xâu; Parity trong tổ hợp và đồ thị; Parity nâng cao. Cuối sách có phần Tổng kết (bản đồ nhận diện trong phòng thi) và Phụ lục gồm 40 bài tập chia thành 10 dạng, mỗi bài có ràng buộc theo subtask như đề thi thật.

Mỗi bài học không đưa ra công thức ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi ví dụ đều được chạy tay trên hình vẽ; mọi chương trình hoàn chỉnh trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Tên biến được viết đầy đủ để em đọc code như đọc một đoạn văn.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: quan sát một thao tác, tìm thông tin tối thiểu, công thức, chạy tay, chứng minh, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Phụ lục và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
CÔNG THỨC	Bất biến, công thức, điều kiện cần và đủ.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là $O(n)$, $O(n^2)$, $O(2^n)$…
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
ĐIỂM DỪNG TƯ DUY	Câu hỏi em nên tự trả lời trước khi đọc tiếp.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Chuỗi tư duy của mọi bài: tình huống → cách ngây thơ → bế tắc → phát hiện thông tin tối thiểu → kiến thức → chứng minh → chạy tay → chương trình → mở rộng.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp khung ĐIỂM DỪNG TƯ DUY, hãy che phần sau lại và tự trả lời trước. Mỗi khi gặp một thao tác mới, hãy tự hỏi ba câu: *một thao tác làm đại lượng nào thay đổi? thay đổi một lượng chẵn hay lẻ? trạng thái đầu và đích có cùng lớp không?* Mục tiêu không phải để nhớ nhiều công thức, mà để khi gặp một đề mới không hề chứa chữ "chẵn" hay "lẻ", em vẫn phát hiện được parity ẩn bên trong.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU 2

CÁCH HỌC VỚI TÀI LIỆU NÀY 3

CHƯƠNG 1. NỀN TẢNG TƯ DUY PARITY 5

Bài 1. Từ giá trị đến một bit: Parity, modulo 2 và XOR 6

Bài 2. Bất biến chẵn lẻ: Tìm điều không đổi thay vì tìm cách làm 14

Bài 3. Parity xen kẽ: Lưới, tô hai màu và đồ thị hai phía 21

CHƯƠNG 2. PARITY TRÊN DÃY VÀ XÂU 29

Bài 4. Prefix parity: Từ hàng tỉ đoạn con đến hai trạng thái 30

Bài 5. XOR và tần suất chẵn lẻ: Điều gì còn lại sau khi triệt tiêu? 36

Bài 6. Parity mask: 26 parity trong một số nguyên 42

Bài 7. Parity động: Cây Fenwick với phép XOR 48

CHƯƠNG 3. PARITY TRONG TỔ HỢP VÀ ĐỒ THỊ 53

Bài 8. Ghép cặp và xây dựng theo lớp chẵn lẻ 54

Bài 9. Bậc của đỉnh: Bổ đề bắt tay và đường đi Euler 59

Bài 10. Tính chẵn lẻ của hoán vị: Swap và nghịch thế 66

CHƯƠNG 4. PARITY NÂNG CAO 72

Bài 11. Quy hoạch động theo parity và trò chơi 73

Bài 12. Ràng buộc XOR và DSU parity 80

Bài 13. GF(2) và khử Gauss: Điểm đến đại số của parity 87

TỔNG KẾT. BẢN ĐỒ NHẬN DIỆN TRONG PHÒNG THI 93

PHỤ LỤC. 40 BÀI TẬP PHÂN THEO 10 DẠNG 96

Lê Thành Nam

CHƯƠNG 1. NỀN TẢNG TƯ DUY PARITY

Có những bài toán mà trạng thái lớn tới 2^{60}, giá trị lớn tới 10^{18}, số đường đi lớn hơn số nguyên tử trong vũ trụ, nhưng câu hỏi cuối cùng chỉ là "có hay không", "chẵn hay lẻ". Chương này dạy em kĩ năng quan trọng nhất của cả cuốn sách: trước khi tối ưu thuật toán, hãy hỏi mình có đang lưu nhiều thông tin hơn mức câu hỏi cần hay không. Em sẽ tự phát hiện ra phép XOR từ bốn phép cộng chẵn lẻ, học cách tìm một đại lượng không đổi để chứng minh "không thể", và nhìn thấy bàn cờ vua ẩn trong mọi bài đi trên lưới.

TRONG CHƯƠNG NÀY

Bài 1. Từ giá trị đến một bit: Parity, modulo 2 và XOR
Bài 2. Bất biến chẵn lẻ: Tìm điều không đổi thay vì tìm cách làm
Bài 3. Parity xen kẽ: Lưới, tô hai màu và đồ thị hai phía

Lê Thành Nam

BÀI 1 Từ giá trị đến một bit: Parity, modulo 2 và XOR

Học xong bài này, em sẽ:

✓ Hiểu parity là một **phép chiếu** trạng thái lớn xuống hai lớp {0, 1}, và biết khi nào phép chiếu đó giữ đủ thông tin.

✓ Tự **suy ra** (không học thuộc) quy tắc $p(a + b) = p(a) \oplus p(b)$, $p(a - b) = p(a) \oplus p(b)$, $p(a \cdot b) = p(a) \wedge p(b)$.

✓ Nắm ba tính chất của XOR: tự triệt tiêu, phần tử trung hòa, kết hợp – giao hoán; biết viết quan hệ "cùng/khác" thành phương trình XOR.

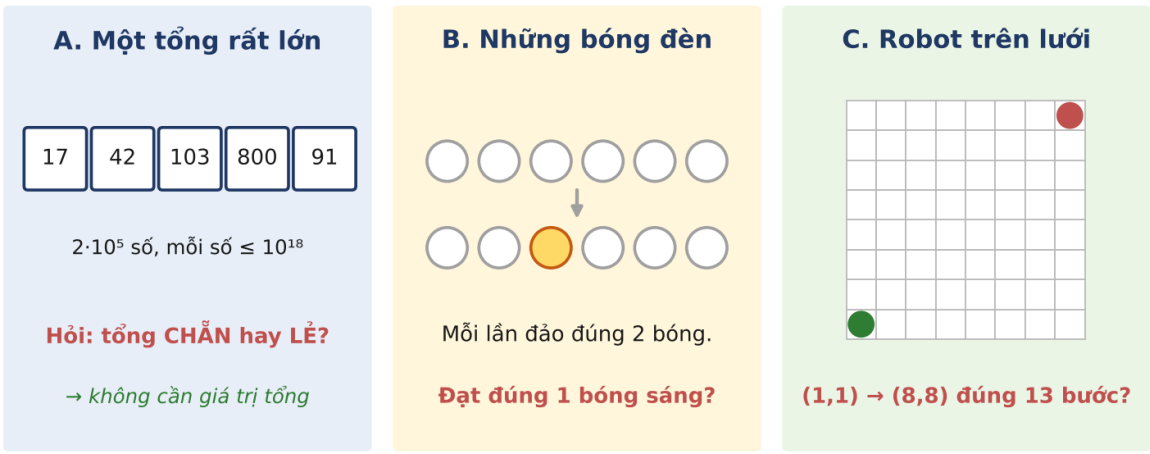
✓ Tránh được các lỗi kinh điển: `x % 2` với số âm, tràn số khi cộng, nhầm thứ tự ưu tiên của `&`.

KHỞI ĐỘNG

Hãy nhìn ba tình huống sau. Chúng được cố ý chọn từ ba "thế giới" khác nhau: một dãy số, một dãy bóng đèn và một bàn cờ.

- Tình huống A. Có 200 000 số nguyên, mỗi số có thể lớn tới 10^{18}. Đề không hỏi tổng bằng bao nhiêu, chỉ hỏi tổng chẵn hay lẻ.
- Tình huống B. Có 60 bóng đèn, ban đầu tất cả tắt. Mỗi thao tác bắt buộc đảo đúng hai bóng. Có thể đạt trạng thái chỉ đúng một bóng sáng không?
- Tình huống C. Robot đứng ở ô (1, 1), mỗi bước sang một ô kề cạnh. Robot có thể tới ô (8, 8) sau đúng 13 bước không?

Ba bài khác nhau, nhưng đều có chung một điều: câu trả lời chỉ có hai khả năng. Nếu em lao vào tính tổng, duyệt trạng thái bóng đèn hay liệt kê đường đi, em đang giải một bài toán lớn hơn bài mà đề thực sự hỏi. Cuối Bài 3, em sẽ trả lời cả ba câu hỏi này trong chưa đầy một dòng lập luận.



Hình 1.1. *Ba bài toán khác nhau nhưng đều chỉ cần phân trạng thái vào* ***hai lớp***

Lê Thanh Nam

BÀI TOÁN. CHẴN HAY LẺ [CHANLE]

Cho dãy n số nguyên $a_1, a_2, \ldots, a_n$. Hãy cho biết tổng $S = a_1 + a_2 + \ldots + a_n$ là số chẵn hay số lẻ.

Dữ liệu vào: Dòng đầu là n ($1 \le n \le 2 \cdot 10^5$). Dòng thứ hai gồm n số nguyên a_i ($|a_i| \le 10^{18}$).

Kết quả: In `EVEN` nếu S chẵn, `ODD` nếu S lẻ.

Dữ liệu vào	Kết quả	Giải thích
5 17 42 103 800 91	ODD	Tổng bằng 1053, là số lẻ.
3 -3 5 -8	EVEN	Tổng bằng −6, là số chẵn (số âm vẫn có thể chẵn).

1. Ý tưởng ngây thơ: Cộng hết rồi kiểm tra

Phản xạ tự nhiên là cộng tất cả phần tử vào một biến `sum`, rồi kiểm tra `sum % 2`. Với ví dụ 1, ta lần lượt tạo ra các tổng 17, 59, 162, 962, 1053 và kết luận 1053 lẻ. Cách này **đúng về mặt toán học**. Nhưng hãy nhìn lại: đề bài không hề hỏi 59, 162 hay 962; nó chỉ hỏi một đáp án hai trạng thái.

Code ý chính: Cộng hết rồi kiểm tra

```
long long sum = 0;
for (int i = 0; i < n; i++) sum += a[i];
if (sum % 2 == 0) cout << "EVEN";
else cout << "ODD";
```

ĐẾM SỐ PHÉP TÍNH

Thời gian O(n): $2 \cdot 10^5$ phép cộng, quá nhanh.

Nhưng độ lớn của tổng: có thể lên tới $2 \cdot 10^5 \times 10^{18} = 2 \cdot 10^{23}$, trong khi `long long` chỉ chứa được tới khoảng $9{,}2 \cdot 10^{18}$. Tổng thật tràn kiểu sau vài chục phép cộng.

Với số nguyên có dấu, tràn số trong C++ là *hành vi không xác định*: chương trình có thể in đúng ở máy em và sai ở máy chấm.

BẾ TẮC

Muốn giữ đúng tổng, ta phải cài đặt số lớn (cộng theo từng chữ số), viết thêm vài chục dòng chỉ để tính một con số có 24 chữ số mà cuối cùng ta chỉ nhìn vào chữ số cuối. Có cách nào không phải giữ tổng không?

Lê Thành Nam

> **ĐIỂM DỪNG TƯ DUY**
> Nếu thay mỗi số bằng đúng một bit 0/1 (chẵn → 0, lẻ → 1), liệu còn đủ thông tin để xác định tổng chẵn hay lẻ không? Nếu có, đó chính là mức thông tin tối thiểu cần giữ. Hãy tự thử với hai số trước khi đọc tiếp.

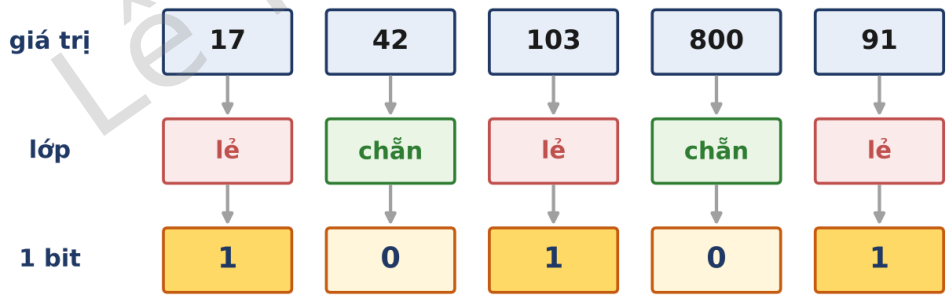
2. Phá bế tắc: Vứt bỏ thông tin thừa

Bước 1 Tự kiểm chứng bốn trường hợp

Ta chưa vội viết công thức. Hãy xét đủ bốn khả năng khi cộng hai số và tự kiểm tra bằng ví dụ cụ thể:

a	b	Ví dụ	a + b	Mã hóa: bit(a), bit(b) → bit(a+b)
chẵn	chẵn	4 + 6 = 10	chẵn	0, 0 → **0**
chẵn	lẻ	4 + 7 = 11	lẻ	0, 1 → **1**
lẻ	chẵn	5 + 6 = 11	lẻ	1, 0 → **1**
lẻ	lẻ	5 + 7 = 12	chẵn	1, 1 → **0**

Hãy che cột cuối và tự điền. Quy luật rất gọn: hai bit giống nhau thì kết quả là 0, khác nhau thì kết quả là 1. Phép toán "giống → 0, khác → 1" có tên là XOR (hoặc loại trừ), kí hiệu $\oplus$, trong C++ viết là `^`. XOR không được "đưa ra" từ trên trời; ta vừa phát hiện nó từ bốn phép cộng chẵn lẻ.



Hình 1.2. Quá trình vứt bỏ thông tin: giá trị → lớp chẵn/lẻ → một bit

Bước 2 Đặt tên cho điều vừa phát hiện

Đến đây mới định nghĩa: **parity** của số nguyên x là $p(x) = x \bmod 2 \in \{0, 1\}$; $p(x) = 0$ nếu x chẵn, $p(x) = 1$ nếu x lẻ. Bảng trên chính là quy tắc cho phép cộng. Phép trừ và phép nhân được suy ra tương tự:

Lê Thành Nam

CÔNG THỨC PARITY

$$p(a + b) = p(a) \oplus p(b)$$

$p(a - b) = p(a) \oplus p(b)$ (vì $-1 \equiv 1 \bmod 2$, trừ cũng như cộng)

$p(a \cdot b) = p(a) \wedge p(b)$ (tích lẻ khi và chỉ khi cả hai thừa số lẻ)

Hệ quả quan trọng nhất: **tổng S lẻ khi và chỉ khi số lượng phần tử lẻ trong dãy là số lẻ.** Mỗi số chẵn đóng góp 0, mỗi số lẻ đóng góp 1 vào XOR cuối cùng.

VÌ SAO ĐÚNG?

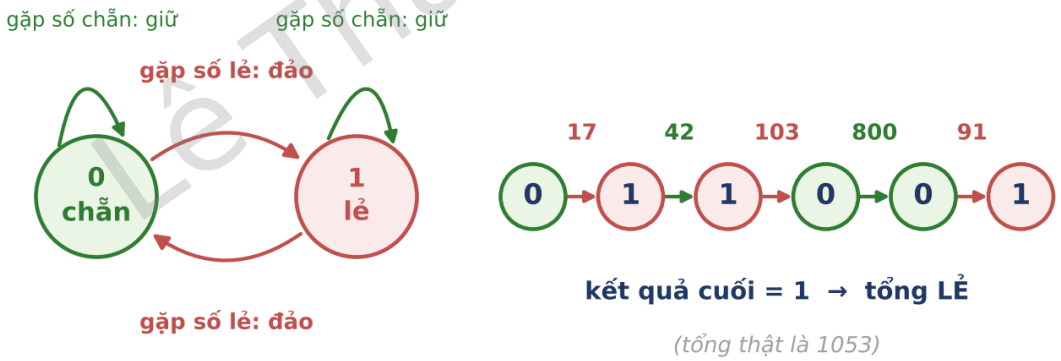
Viết $a = 2q + r$, $b = 2s + t$ với $r, t \in \{0, 1\}$. Khi đó $a + b = 2(q + s) + (r + t)$. Phần $2(q + s)$ luôn chẵn, nên parity của $a + b$ chỉ phụ thuộc vào $r + t \in \{0, 1, 2\}$: bằng 1 khi đúng một trong r, t bằng 1, tức là $r \oplus t$. Tương tự, $a \cdot b = 2(\ldots) + r \cdot t$, nên parity của tích là $r \cdot t = r \wedge t$.

Áp dụng lặp lại cho n số: $p(a_1 + \ldots + a_n) = p(a_1) \oplus \ldots \oplus p(a_n)$. XOR của n bit bằng 1 khi và chỉ khi có lẻ bit 1, tức là có lẻ số lẻ.

Lê Thành Nam

Bước 3 Chạy tay: Một máy trạng thái hai ô

Với dãy 17, 42, 103, 800, 91, ta chỉ giữ một bit `parity`, ban đầu bằng 0 (tổng rỗng bằng 0, là số chẵn). Gặp số chẵn: giữ nguyên. Gặp số lẻ: đảo bit.



Hình 1.3. *Máy trạng thái hai ô: số chẵn giữ nguyên trạng thái, số lẻ đảo trạng thái*

Bước	a_i	a_i lẻ?	parity trước	parity sau	Tổng thật (để đối chiếu)
1	17	có	0	**1**	17
2	42	không	1	**1**	59
3	103	có	1	**0**	162
4	800	không	0	**0**	962
5	91	có	0	**1**	1053

Cột "parity sau" luôn khớp với tính chẵn lẻ của cột tổng thật, nhưng ta **không bao giờ phải giữ cột tổng thật**. Không còn tràn số, không cần số lớn.

Code ý chính: Chỉ giữ một bit

```
int parity = 0;
for (int i = 0; i < n; i++) {
    long long a;
    cin >> a;
    parity ^= (int)(a & 1);     // a & 1 là bit cuối của a
}
```

3. Chương trình hoàn chỉnh

Phần đọc tệp được viết theo kiểu "có tệp thì đọc tệp": khi chấm bằng Themis với tệp CHANLE.INP, chương trình đọc và ghi tệp; khi nộp lên trang chấm trực tuyến, điều kiện `fopen` sai nên chương trình đọc chuẩn như bình thường.

CHANLE.CPP: Một bit trạng thái, O(n) thời gian, O(1) bộ nhớ

```
#include <cstdio>
#include <iostream>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("CHANLE.INP", "r")) {
        freopen("CHANLE.INP", "r", stdin);
        freopen("CHANLE.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    // 0: tổng đang chẵn, 1: tổng đang lẻ
    int parity = 0;
    for (int i = 0; i < n; i++) {
        long long a;
        cin >> a;
        // số lẻ đảo trạng thái, số chẵn giữ nguyên
        parity ^= (int)(a & 1);
    }
    cout << (parity ? "ODD" : "EVEN") << '\n';
    return 0;
}
```

Lê Thanh Nam

Vài chi tiết đáng chú ý:

Biểu thức `a & 1` lấy bit cuối cùng của a. Với số âm biểu diễn bù hai (mọi máy chấm hiện nay), `-3 & 1` bằng 1 và `-8 & 1` bằng 0, đúng như mong muốn. Ngược lại, `-3 % 2` trong C++ cho kết quả -1, không phải 1.

Độ phức tạp vẫn là O(n) như cách ngây thơ, nhưng **tư duy đã thay đổi hoàn toàn**: ta không còn duy trì một giá trị lớn mà duy trì đúng **một bit trạng thái**. Ở các bài sau, chính sự thay đổi tư duy này (chứ không phải tốc độ) mới đem lại những bước nhảy từ $O(2^n)$ xuống O(1).

4. Modulo 2 và XOR: Ngôn ngữ tự nhiên của parity

Nếu chỉ nhớ $p(a + b) = p(a) \oplus p(b)$, em sẽ dùng được vài bài rồi quên. Ta cần nhìn sâu hơn một chút. Khi chỉ quan tâm số dư cho 2, thế giới số nguyên co lại thành hai phần tử 0 và 1, với một quy tắc lạ mà rất quan trọng:

$$1 + 1 \equiv 0 \pmod 2$$

Vì vậy **phép cộng modulo 2 chính là XOR**. Mọi bài toán chỉ quan tâm tổng modulo 2 đều có thể chuyển sang ngôn ngữ XOR, và XOR có ba tính chất mà em phải hiểu bằng trực giác:

Tính chất	Công thức	Ý nghĩa trực giác	Sẽ dùng ở
Tự triệt tiêu	$x \oplus x = 0$	Một thông tin xuất hiện hai lần thì biến mất	Bài 5 (tần suất)
Phần tử trung hòa	$x \oplus 0 = x$	Thêm "không có gì" không làm thay đổi	Bài 4 ($P_0 = 0$)
Kết hợp, giao hoán	$a \oplus b \oplus c = c \oplus (a \oplus b)$	Thứ tự và cách nhóm không quan trọng	Bài 5, 7, 13
Tự nghịch đảo	$a \oplus b = c \Leftrightarrow a = b \oplus c$	"Trừ" cũng là XOR, không cần phép chia hay phép trừ riêng	Bài 4, 12, 13

Từ "cùng/khác" đến một phương trình

Giả sử hai đối tượng u, v mỗi cái có một trạng thái $x_u, x_v \in \{0, 1\}$ (bật/tắt, đội A/đội B, trắng/đen). Hai câu nói "u và v **cùng** trạng thái" và "u và v **khác** trạng thái" được viết chung bằng **một** phương trình:

$$\mathbf{x_u \oplus x_v = w},$$ với $w = 0$ nghĩa là cùng, $w = 1$ nghĩa là khác.

Phương trình nhỏ bé này sẽ trở thành cạnh của đồ thị tô hai màu ở Bài 3, cạnh có trọng số trong DSU parity ở Bài 12 và một hàng của ma trận ở Bài 13.

Parity không phải thuộc tính của dữ liệu: ta chủ động chọn nó

Ta có thể lấy parity của **bất kì** đại lượng nguyên nào trong bài: tổng, số phần tử được chọn, số lần thao tác, số bước đi, tọa độ $x + y$, số nghịch thế của một hoán vị, bậc của một đỉnh, số bit 1 của một số, tần suất của một kí tự… Đây là bước chuyển quan trọng nhất của cả cuốn sách:

> **PARITY LÀ MỘT PHÉP CHIẾU TA CHỦ ĐỘNG CHỌN**
>
> Khi một thao tác phức tạp diễn ra, hãy hỏi: *nếu chỉ nhìn một đại lượng phù hợp theo modulo 2, thao tác ấy có trở nên đơn giản hơn không?* Từ Bài 2 trở đi, chìa khóa thường nằm ở parity của một **đại lượng phụ** mà đề không hề nhắc tới, chứ không phải parity của giá trị trong input.

Lê Thành Nam

5. Mở rộng

Mở rộng 1: Tràn số không đáng sợ nếu chỉ cần parity

Với kiểu **không dấu** `unsigned long long`, phép cộng tự động lấy dư cho 2^{64} và tràn số là hợp lệ. Vì 2^{64} chia hết cho 2, việc lấy dư này **không làm thay đổi bit cuối**. Vậy nếu đọc vào kiểu không dấu rồi cộng thoải mái, bit cuối của tổng vẫn đúng. Đây là ví dụ đẹp của nguyên lí: *phép chiếu modulo 2 tương thích với mọi phép chiếu modulo 2^k*.

Mở rộng 2: Parity của số bit 1

Parity của số bit 1 trong biểu diễn nhị phân của x bằng XOR của mọi bit của x. GCC có sẵn hàm `__builtin_parityll(x)` trả về giá trị này trong O(1). Ví dụ $13 = 1101_2$ có ba bit 1 nên parity bằng 1.

Mở rộng 3: Tổng 1 + 2 + … + n chẵn khi nào?

Trong 1, 2, …, n có $\lceil n/2 \rceil$ số lẻ. Tổng lẻ khi và chỉ khi $\lceil n/2 \rceil$ lẻ, tức là $n \bmod 4 \in \{1, 2\}$. Đây là dạng câu hỏi "chia 1..n thành hai nhóm có tổng bằng nhau" rất hay gặp: điều kiện cần là tổng chẵn, tức $n \bmod 4 \in \{0, 3\}$.

Mở rộng 4: Khi nào nén xuống một bit là SAI?

Parity chỉ đủ khi **mọi quyết định tiếp theo** của bài chỉ phụ thuộc vào parity. Nếu đề hỏi tổng chia hết cho 3, hay tổng có lớn hơn K không, thì việc nén xuống một bit đã làm mất thông tin cần thiết. Bài 11 sẽ cho em một tiêu chuẩn chính xác để kiểm tra điều này.

BẪY THƯỜNG GẶP

`x % 2 == 1` sai với số âm: `-3 % 2` bằng −1. Hãy dùng `x % 2 != 0` hoặc `(x & 1)`.

Thứ tự ưu tiên toán tử: `if (x & 1 == 0)` được hiểu là `x & (1 == 0)`, luôn sai. Phải viết `if ((x & 1) == 0)`.

Khai báo `int` cho a_i lớn tới 10^{18}: dữ liệu đọc vào đã sai trước cả khi lấy parity.

Nhầm parity của giá trị với parity của số lượng, của vị trí, của số bước. Câu hỏi đầu tiên luôn là: *ta đang lấy parity của đại lượng nào?*

GHI NHỚ

Parity = phép chiếu từ trạng thái lớn xuống {0, 1} (chẵn → 0, lẻ → 1).

Cộng, trừ modulo 2 là XOR; nhân modulo 2 là AND. Vì $1 + 1 \equiv 0$.

Tổng lẻ ⇔ có lẻ số hạng lẻ.

XOR: tự triệt tiêu, trung hòa với 0, kết hợp – giao hoán, tự nghịch đảo.

Quan hệ cùng/khác ⇔ phương trình $x_u \oplus x_v = w$.

Câu hỏi then chốt: *Mình có đang giữ nhiều thông tin hơn mức câu hỏi cần không?*

Lê Thành Nam

LUYỆN TẬP

Bài	Tên	Gợi ý liên hệ
A1	CẶP CHẴN	Chia dãy thành các cặp có tổng chẵn. Viết bảng parity của một cặp trước khi nghĩ tới thuật toán ghép.
D1	SỐ CÔ ĐỘC	Tìm số xuất hiện lẻ lần. Tính chất "tự triệt tiêu" của XOR.
F1	ĐÚNG K BƯỚC	Dạo đầu cho Bài 3: parity của số bước đi trên lưới.

Tự kiểm tra nhanh

1. Dãy có 7 số lẻ và 100 số chẵn. Tổng của dãy chẵn hay lẻ? *(Đáp số: lẻ, vì có lẻ (7) số lẻ)*
2. Tính parity của $3^{2026} + 2^{2026}$. *(Đáp số: 1 (lẻ): 3^{2026} lẻ, 2^{2026} chẵn)*
3. Cho a lẻ, b chẵn, c lẻ. Tính p(a·b + c) và p(a·c − b). *(Đáp số: $p(a\cdot b + c) = 0 \oplus 1 = 1$; $p(a\cdot c - b) = 1 \oplus 0 = 1$)*
4. Với n nào thì tổng 1 + 2 + … + n chẵn? Thử với n = 2026. *(Đáp số: $n \bmod 4 \in \{0, 3\}$; $2026 \bmod 4 = 2$ nên tổng lẻ)*

Lê Thành Nam